DisasterRes-Net: Classification Improvement Spec

Prepared for Christopher, Parallax Machine Inc. Scope: the disaster_type (peril) classifier first, then severity, then the optional SegFormer module.

1. Problem statement

The current model (frozen Inception-ResNet-V2, 2036-D fused features, Random Forest) reaches 68.45% accuracy on 5,433 test images, but macro recall is only 35.7% and macro F1 is 40.2%. It behaves almost like a two-class model: 96% of predictions are Hurricane or Not Disaster.

Peril	Support	Precision	Recall
Earthquake	387	90.6%	19.9%
Flood	96	100%	2.1%
Hurricane	2,060	64.2%	68.5%
Landslide	28	100%	7.1%
Not Disaster	2,553	70.0%	83.2%
Wildfire	309	92.0%	33.3%

Key observations driving this spec:

- Rare classes have high precision and very low recall, so the decision rule is the first suspect, not the features.
- 908 of 2,880 real disaster images (31.5%) are predicted Not Disaster. This is the costliest error for an emergency system.
- Hurricane and Not Disaster confusion is 1,062 of 1,714 errors (62%). Label noise must be checked.
- 5-fold stratified CV on 12,670 training images gives 68.06% +/- 1.06%. This confirms a stable estimate, but not a good model.

2. Goals, non-goals, success metrics

Goals

- Raise macro F1 and per-class recall for rare perils without collapsing Hurricane and Not Disaster precision.
- Cut the disaster-called-Not-Disaster miss rate.
- Produce calibrated probabilities with an explicit abstain path.
- Keep Flask and CrewAI integration working throughout.

Non-goals (this spec)

- Replacing the CrewAI agent layer.
- SAR and multispectral fusion, bi-temporal Siamese networks.
- SegFormer is specified in Phase 9 as a separate, deferred module. Switching backbone (ConvNeXt, EfficientNetV2, DINOv2) is also deferred; this spec uses Inception-ResNet-V2 throughout.

Success metrics (proposed targets, to be revised after Phase 0)

- Headline metrics: macro F1, balanced accuracy, per-class recall. Accuracy is secondary.
- Phase 1 target: macro F1 at or above 55%, Flood and Landslide recall at or above 30%.
- Phase 3 target: macro F1 at or above 70%, every class recall at or above 50%, disaster-to-Not-Disaster miss rate at or below 15%.
- Every number is reported with fold-level spread and, for classes under 100 samples, a confidence interval.

3. Evaluation protocol (applies to every phase)

1. The 5,433-image test set is touched once per candidate model, after all tuning is finished.
2. All tuning (thresholds, class weights, hyperparameters, early stopping) uses out-of-fold predictions on the 12,670 training images via cross_val_predict with the existing StratifiedKFold(5, shuffle=True, random_state=42).
3. Scorers: balanced_accuracy and f1_macro in addition to accuracy. Log per-class precision, recall and F1 per fold.

4. Log to MLflow: cv_f1_macro_mean/std, cv_balanced_acc_mean/std, per-class recall, test confusion matrix as an artifact, class counts, git commit, feature spec version.
5. Any scaler or feature selection is fitted inside each fold (sklearn Pipeline), never on the full training set.
6. Report the confusion matrix, not only scalar metrics, for every comparison.

Phase 0: Diagnostics and data audit

No model changes. Purpose: find out how much of the error is model error versus data error.

1. Train-vs-held-out gap: compute clf.score(x_train, y_train) and compare with 0.68. A near-100% train score confirms the Random Forest memorizes.
2. Class counts: print train/test counts per peril and per severity. Record imbalance ratios (Hurricane to Landslide is about 73:1 in test).
3. Duplicate and leakage check: compute perceptual hashes (pHash/dHash) or CNN embedding cosine similarity across train and test. Report the number of near-duplicate pairs crossing splits and remove or regroup them. Use GroupKFold on duplicate clusters if many exist.
4. Label audit: sample 50 images from each of: Hurricane predicted Not Disaster (643), Not Disaster predicted Hurricane (419), Earthquake predicted Hurricane (183), Wildfire predicted Hurricane (111). Hand-label as model-wrong, label-wrong, or ambiguous. Record the percentages.
5. Image-source audit: tag images as overhead, ground-level, or screenshot/graphic (manual on the samples above). This sets how much a perspective gate matters later.
6. Deliverable: a short findings note with the label-noise estimate, duplicate count and per-class counts. If label noise exceeds about 20% of the audited errors, add a label-cleaning step before Phase 3.

Phase 1: Fix the decision rule on the existing Random Forest

Goal: recover rare-class recall cheaply without changing features.

1. Retrain with class_weight='balanced_subsample' and min_samples_leaf between 3 and 5 (grid: 1, 3, 5, 10) and max_features in {sqrt, 0.1}. Fully grown trees give near-zero probabilities for rare classes on unseen data.
2. Keep n_estimators at 300 (check 500 only if OOF macro F1 still improves).

3. Generate out-of-fold probabilities with cross_val_predict(method='predict_proba') on all 12,670 training images.
4. Fit per-class probability multipliers w_k (prediction = argmax of P times w) by coordinate search maximizing OOF macro F1. Bound w_k to [0.5, 20]. Alternative: prior correction by dividing by class frequency raised to a power tau, with tau tuned on OOF.
5. Add a cost-sensitive variant: penalize the disaster-to-Not-Disaster error more (for example, cost 2 to 3 times) and compare. This is a policy decision for you, so report both operating points.
6. Freeze w_k inside the saved artifact bundle and apply it once to the test set. Report the new confusion matrix against the baseline.
7. Acceptance: macro F1 up at least 10 points over 40.2%, no class recall below its baseline by more than 5 points except Not Disaster precision, which may trade off.

Phase 2: Feature upgrade and ablation

Goal: replace weak features and find out what actually helps.

1. First confirm preprocessing: the timm inception_resnet_v2 weights expect 299x299 input with mean and std of 0.5 per channel, not the usual ImageNet statistics. Compare with what step3_classify.py does today and build transforms from timm.data.resolve_data_config so training and inference match. Then extract pooled features instead of ImageNet logits: create the model with num_classes=0 to get 1536-D features (the current 1000-D vectors are classifier logits).
2. Build a new feature spec (for example v3) and register it in resolve_artifact_spec so the 2030-D and 2036-D artifacts still load. Version the .npz cache filename with the spec name.
3. Ablation grid on OOF macro F1 (same Random Forest settings from Phase 1):
   - df1 only; df1 + df2; df1 + glcm + lbp; all four;
   - 1000-D logits vs 1536-D pooled vs 1536-D pooled plus the pooled output of mixed_6a (1088-D, keeps more texture detail than the final layer).
4. Optional and deferred: a DINOv2 or CLIP linear probe as a reference point for the rare classes (Flood, Landslide, Earthquake, Wildfire). Run it only if Inception-ResNet-V2 results plateau.
5. Decision rule: keep the cheapest feature set whose OOF macro F1 is within 1 point of the best. Drop saliency, GLCM or LBP if they do not help; this also removes the LMS saliency compute from inference.

6. Deliverable: ablation table in MLflow plus the chosen feature spec.

Phase 3: End-to-end fine-tuning (enhancement 2)

3.1 Model

- Backbone: Inception-ResNet-V2 (timm inception_resnet_v2, about 56M parameters, 299x299 input, 1536-D pooled output). Alternative backbones are deferred until its results are in.
- Shared backbone, three heads: peril (6 classes), severity (3 classes), informativeness (2 classes). Heads are a dropout layer (0.3 to 0.5) plus linear (optionally one hidden layer of 512); use label smoothing of 0.05 to 0.1. The original and saliency images pass through the same backbone weights; Phase 2 decides whether the saliency stream is kept.
- If Phase 2 shows handcrafted features help, concatenate GLCM/LBP to the pooled vector before the heads; if the mixed_6a ablation wins, concatenate its pooled 1088-D vector as well.

3.2 Loss and sampling (choose one imbalance mechanism per head)

- Default: logit-adjusted cross-entropy (add log prior to logits during training) or focal loss with gamma 2, with alpha from effective-number class weights (beta 0.999).
- Do not stack focal loss, class weights and a balanced sampler at the same time. Run these arms and compare: (a) focal + effective-number weights; (b) logit-adjusted CE; (c) class-balanced sampler + plain CE.
- Multi-task loss: L = L_peril + 0.5 L_severity + 0.25 L_info as a starting point; tune on OOF.
- Severity and informativeness labels may be missing or imbalanced per peril; mask missing labels.

3.3 Unfreezing schedule

- Stage A (3 to 5 epochs): backbone frozen, train heads only, LR 1e-3.
- Stage B (5 to 10 epochs): unfreeze mixed_7a, repeat_2 (the nine Block8 units), block8 and conv2d_7b; backbone LR 1e-5, head LR 1e-3. Confirm the layer names with named_children() first, since timm names can change between versions.

- Stage C (optional): add mixed_6a and the upper half of repeat_1 (the twenty Block17 units, indices 10 to 19); backbone LR 5e-6, only if validation macro F1 still improves after stage B. Use layer-wise LR decay (about 0.8 per stage).
- Optimizer AdamW, weight decay 1e-2, cosine schedule with 1-epoch warmup, gradient clipping 1.0, mixed precision on GPU.
- BatchNorm: keep BN layers in eval mode (or freeze their statistics) when the batch size is below 32, and re-apply eval mode after every model.train() call.

3.4 Data pipeline

- Input: 299x299 with mean and std of 0.5 per channel (timm's config for this model); try 384 only if small damage regions matter, since the pretrained weights expect 299.
- Augmentation: random resized crop (scale 0.6 to 1.0), horizontal flip, mild color jitter, random erasing. No vertical flip for ground-level images. Avoid augmentations that change the texture statistics (heavy blur) because texture is a signal.
- The .npz feature cache no longer applies; cache decoded, resized images to disk instead.

3.5 Training and validation

- Reuse the same 5-fold splits. Train on 4 folds, validate on 1, early stop on validation macro F1 (patience 3). Report fold mean and spread.
- After picking hyperparameters, train a final model on all 12,670 training images for the epoch count found in CV, and evaluate once on the test set.
- Seeds: fix and log; run 3 seeds for the final candidate to report variance.
- Hardware: use mixed precision, channels_last and gradient accumulation (effective batch 32) for this roughly 56M-parameter model; record GPU, batch size, epochs and wall-clock; set a budget before starting.

3.6 Acceptance

- Beats the Phase 1 tuned Random Forest on OOF macro F1 by at least 5 points and on the test set by a margin larger than the seed variance. If not, ship the Phase 1/2 model and stop here.

Phase 4: Peril-conditioned severity head

Do this only after peril recall is acceptable (every class at or above 50%).

1. Replace the single severity head with K peril-specific heads (K = 5 perils plus not_disaster if severity applies).
2. Combine softly: severity logits = sum over k of p(peril = k) times head_k(features). Train jointly with the peril head, using true peril as a teacher-forcing mixture for the first epochs, then switch to predicted probabilities.
3. Heads with very few samples (Landslide, Flood) share a low-rank adapter or fall back to the shared severity head.
4. Compare with the shared head on OOF severity macro F1 and on DEM agreement. Keep only if it wins.
5. This replaces the router-plus-specialist and mixture-of-experts designs. Reasons: soft routing avoids cascaded errors, and per-peril data stays pooled instead of split.

Phase 5: Calibration, abstention and test-time augmentation

1. Fit temperature scaling per head on OOF logits (a single scalar per head). Report expected calibration error before and after.
2. Abstain rule: if max calibrated probability is below a per-class threshold, or the top-two margin is below a margin threshold, return 'uncertain' with the top-2 classes. Tune thresholds on OOF to hit a chosen operating point (for example, at least 90% precision on answered images, abstention at most 15%).
3. Replace the idea of running the model twice with test-time augmentation: average probabilities over horizontal flip plus 2 to 3 crop scales. Measure the gain on OOF; keep only if it helps and the latency fits.
4. Add an out-of-distribution guard: Mahalanobis or max-softmax score on embeddings to flag images unlike training data (screenshots, memes).

Phase 6: Data expansion for rare classes

1. Target Flood, Landslide, Earthquake and Wildfire. Aim for at least 300 labeled images per class so test metrics are meaningful (currently 96 flood and 28 landslide in test).
2. Candidate sources: public disaster image datasets with suitable licenses, ReliefWeb/GDACS imagery, your own scraped sets (log source and license per image).
3. Hold out a fresh, source-disjoint evaluation set for rare classes so duplicates and source bias cannot inflate scores.
4. Optional: synthetic augmentation (copy-paste, style variation) only after real data is exhausted, and never in validation or test.

Phase 7: Integration

1. Artifact bundle: store model weights, class order, calibration temperatures, per-class thresholds, feature spec version and training metadata together. Extend resolve_artifact_spec to validate them so old Random Forest artifacts keep loading (LRU cache of size 3 stays).
2. Pipeline singleton: load the new model in pipeline_singleton.py behind the same bounded ThreadPoolExecutor(max_workers=2); add a config flag to switch between legacy RF and the new model for rollback.
3. API: extend POST /api/predict with calibrated probabilities, an 'uncertain' flag and the top-2 classes. Keep existing fields so templates/index.html does not break.
4. DEM: keep the current formula but feed it calibrated severity confidence instead of raw RF confidence; document the change because DEM values will shift.
5. CrewAI and the numeric fidelity gate: add the new fields (abstain flag, calibrated confidences) to FactsBlock so the report text can mention uncertainty. When the model abstains, the Analyst agent requests another image instead of asserting a peril.
6. Confidence bands (High, Medium, Moderate): re-derive from calibrated probabilities and abstention, not from raw RF scores.
7. Regression tests: golden-image tests for each peril, a test that legacy artifacts still load, and a latency check on CPU.

Phase 8: Reporting and release gates

- Produce a single comparison table across: baseline RF, Phase 1 tuned RF, Phase 2 best features, Phase 3 fine-tuned, plus Phase 5 calibrated and abstaining. Columns: macro F1, balanced accuracy, per-class recall, disaster-miss rate, latency, model size.
- Release gate: no class recall below 50%, disaster-to-Not-Disaster miss rate at or below 15%, calibration ECE at or below 0.08, latency within the budget you set for Flask and SSE flows.
- Update the README or learnings.md with the new evaluation protocol so accuracy is no longer the headline.

Phase 9 (deferred): SegFormer damage segmentation (enhancement 1)

Start only after Phases 0 to 5 are done. It improves localization and DEM for overhead imagery; it does not fix peril classification.

1. Prerequisite: a perspective gate (overhead vs ground-level vs graphic) using the Phase 0 image-source labels. Ground-level uploads skip segmentation and fall back to Grad-CAM localization.
2. Data: xBD for building damage (check its license, which I believe is non-commercial, before any commercial use). For flood, landslide and burn extent, evaluate Sen1Floods11, FloodNet, Landslide4Sense or similar, since xBD does not cover them well.
3. Model: SegFormer B3 or Mask2Former, 4-class damage output (no-damage, minor, major, destroyed). Use pre/post pairs if the data allows; otherwise post-only.
4. Metrics: per-class IoU and F1 on building damage, plus agreement between the new mask and the old Grad-CAM mask.
5. Integration: new tool next to DamageAssessmentTool; DEM variant using building-level damage fractions; keep the old path as fallback.

Risks and mitigations

- Label noise caps achievable accuracy: Phase 0 audit, clean labels before fine-tuning.
- Rare-class test sets are tiny (28 landslides): use OOF evaluation, confidence intervals, and a fresh source-disjoint set (Phase 6).
- Fine-tuning may not beat a tuned RF on this data size: Phase 3 has an explicit stop condition.
- Overfitting threshold tuning to the test set: thresholds fit on OOF only.
- Duplicate leakage inflates every score: Phase 0 check and GroupKFold.
- DEM drift after model change: version the DEM and report the shift.
- Dataset license issues (xBD, scraped images): record source and license per image.

Suggested order and dependencies

1. Phase 0, then Phase 1 (days, no new compute).
2. Phase 2 (feature extraction compute, one run per backbone).

3. Phase 3 (needs GPU), then Phase 5.
4. Phase 6 runs in parallel with 2 to 5.
5. Phase 4 after Phase 3 passes its acceptance.
6. Phase 7 and 8 after the winning model is chosen.
7. Phase 9 last.